



Samuel Fostiné

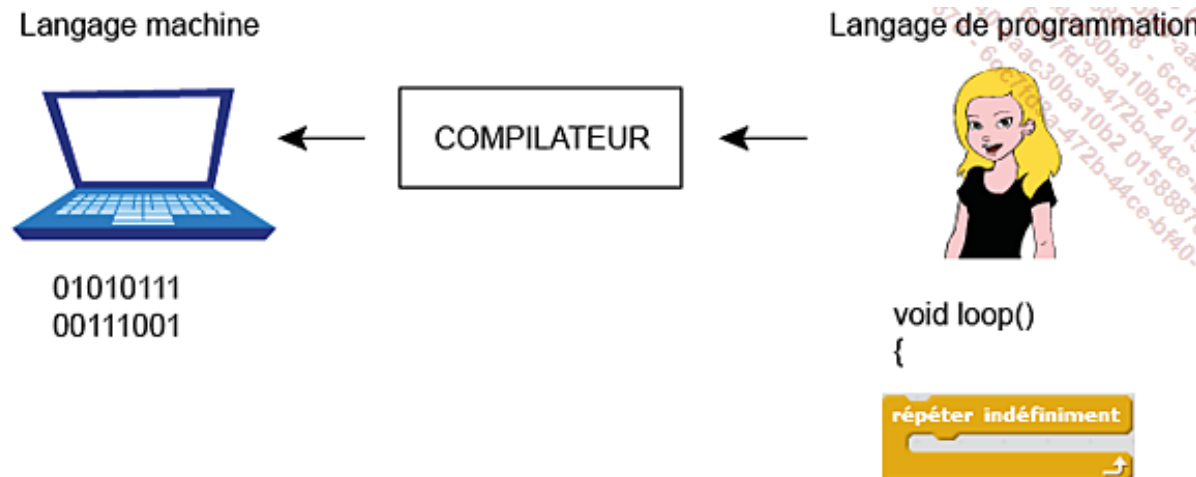
Introduction à la programmation informatique avec Scratch 3.0

Automne 2025

Cette présentation est inspirée du livre :

Boccara, V. (2023). Scratch 3 - S'initier à la programmation, à la robotique et à l'IA par le jeu (2e éd.).
Éditions ENI

-  Sans logiciel, un ordinateur n'est qu'une enveloppe vide.
-  Un logiciel correspond à une suite d'instructions que la machine peut exécuter.
-  Le langage compris par l'ordinateur est le langage binaire, constitué uniquement de 0 et de 1.
-  Pour donner des consignes à l'ordinateur, le développeur utilise un langage de programmation.
-  Ce langage de programmation est plus proche de celui des humains puisqu'il est formé de mots et de symboles.
-  Afin d'être interprété et exécuté par l'ordinateur, le programme doit être converti en langage machine grâce à un compilateur.



Programmer avec Scratch

-  Scratch est un langage visuel basé sur des blocs qui s'assemblent par glisser-déposer. Ces blocs, organisés en catégories, s'empilent comme des Lego pour créer des programmes sans écrire de code.
-  Avec ses blocs colorés, Scratch facilite la programmation et permet de créer dès la première utilisation.
-  Scratch introduit les notions essentielles de la programmation : variables, conditions, boucles, événements.
-  Scratch constitue une première étape avant de passer à des langages plus complexes comme Java ou Python.
-  Scratch est largement utilisé pour développer des jeux vidéo, des animations et des histoires interactives.

L'interface de Scratch 3

The image shows the Scratch 3 web interface. At the top is a purple menu bar with icons for 'Scratch', 'Paramètres', 'Fichier', 'Modifier', 'Untitled', 'Partager', 'Voir la page du projet', 'Tutoriels', 'Debug', 'Enregistrer maintenant', and a user profile 'footsam'. Below the menu bar is a light blue bar with tabs for 'Code', 'Costumes', and 'Sons'. The main workspace is divided into three sections: a left sidebar for block palettes, a large central area for script building, and a right sidebar for scene and sprite management.

Barre de menus: The top purple bar containing navigation and utility icons.

Code: The left sidebar containing categorized block palettes: Mouvement, Apparence, Son, Événements, Contrôle, Capteurs, Opérateurs, Variables, and Mes Blocs.

Espace des scripts: The large central workspace with a grid background for assembling code blocks.

Scène: The right sidebar's top section, showing the current scene with the Scratch cat sprite.

Fenêtre des sprites: The bottom right section, showing the 'Sprite1' window with settings for x, y, Taille, and Direction.

Palette des blocs: A label pointing to the 'Code' sidebar.

Sac à dos: A label at the bottom center of the interface.

L'interface de Scratch 3

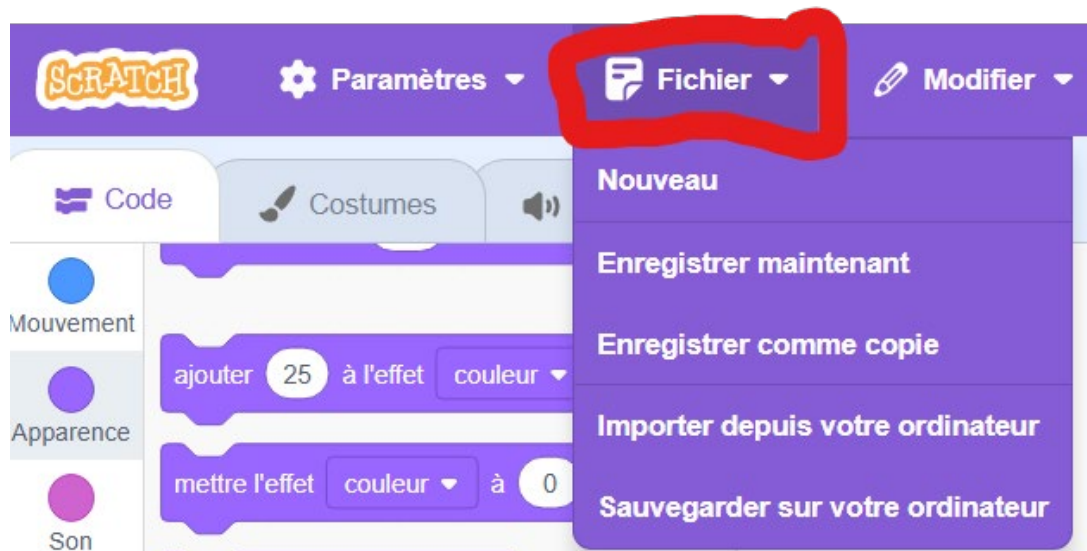
-  L'interface de Scratch 3 est colorée, simple et pensée pour être facile à utiliser.
-  La barre de menus est placée en haut de l'écran, elle permet d'accéder aux principales options du logiciel.
-  La palette des blocs est située à gauche, elle regroupe tous les blocs classés par catégories pour créer des programmes.
-  L'espace des scripts se trouve au centre. On y trouve la zone de programmation où les blocs sélectionnés viennent s'assembler.
- À droite, on retrouve :
 - La scène où le projet s'exécute.
 - La fenêtre des sprites qui contient les personnages et objets.
 - La fenêtre des arrière-plans qui gère les décors du projet.

La barre de menu

- Placée en haut de l'écran, elle est composée
 - Des icônes suivantes:
 - Scratch qui est un raccourci vers la page d'accueil
 - Les paramètres contenant une option pour changer la langue et le mode de couleur de l'écran



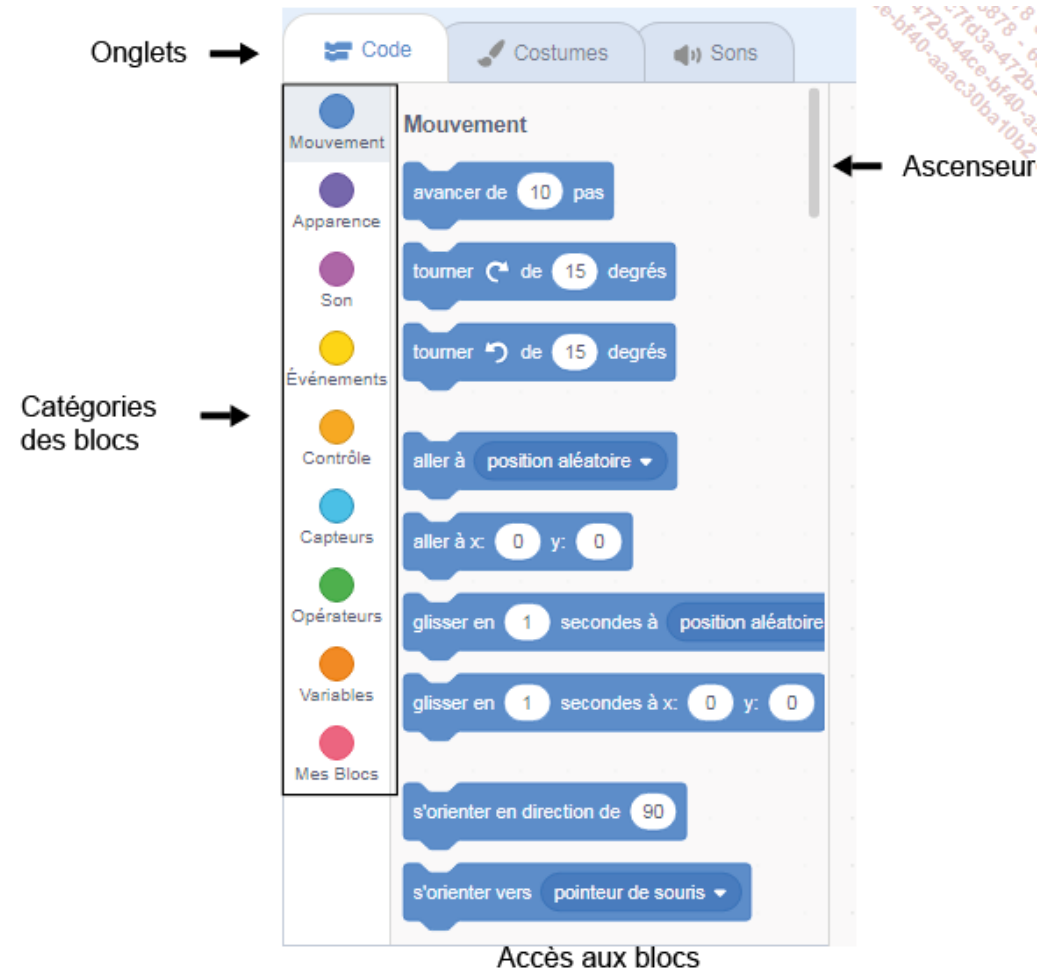
Le menu fichier



- Nouveau : crée un projet vide pour repartir de zéro.
- Importer : ouvre un projet déjà enregistré sur l'ordinateur.
- Sauvegarder sur l'ordinateur : enregistre une copie locale du fichier .sb3.
- Enregistrer maintenant : sauvegarde le projet en ligne dans votre compte Scratch.
- Enregistrer comme copie : duplique le projet pour conserver différentes versions.

La palette des blocs

- Située à gauche de l'interface, la **palette des blocs** est composée de trois onglets.
- **Code** : les blocs pour créer ton programme.
- **Costumes / Arrière-plans** : pour changer et dessiner l'apparence des personnages ou du décor.
- **Sons** : pour ajouter ou modifier des bruits et de la musique.



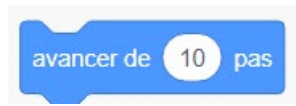
Catégories de blocs (onglet code)

L'onglet **Code** contient plus de cent blocs de programmation, organisés en 9 catégories de couleurs différentes pour mieux les reconnaître:

1. **Mouvement (bleu foncé)** : permet de déplacer les personnages dans tous les sens sur la scène.
2. **Apparence (violet)** : sert à modifier l'aspect du personnage, le faire parler, disparaître ou appliquer des effets visuels.
3. **Son (mauve)** : permet d'ajouter des bruitages, de la musique et de gérer le volume.
4. **Événements (jaune)** : déclenche des actions quand un événement se produit, comme cliquer sur le drapeau vert ou une touche du clavier.
5. **Contrôle (orange-jaune)** : gère l'exécution du programme en ajoutant des conditions, des boucles ou des pauses.
6. **Capteurs (turquoise)** : détecte ce qui se passe (par exemple la position de la souris ou une touche pressée) et crée des interactions.
7. **Opérateurs (vert)** : réalise des calculs, compare des valeurs ou travaille sur du texte.
8. **Variables (orange)** : sert à stocker et utiliser des données comme un score, un temps ou une liste d'éléments.
9. **Mes blocs (rose)** : permet de créer ses propres blocs personnalisés pour réutiliser des séquences de code.

Les blocs Mouvement

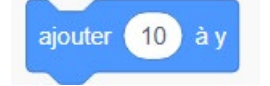
- Les blocs de la catégorie Mouvement permettent de contrôler la position et les déplacements d'un lutin sur la scène, en précisant son orientation ou sa place par rapport aux autres personnages.
- Les déplacements relatifs permettent de bouger un lutin par rapport à sa propre position actuelle ou par rapport à un autre lutin, sans utiliser les coordonnées de la scène. Dans Scratch, trois blocs servent à réaliser ce type de mouvement en indiquant un nombre de pas (par défaut 10), qui correspond à l'unité de déplacement sur la scène.



Le lutin se déplace vers l'avant selon le nombre de pas indiqué. Si la valeur est négative, le déplacement se fait en arrière, par exemple avancer de -10 pas permet de reculer.



Ce bloc modifie la position x du lutin de la valeur spécifiée. Le lutin se déplace horizontalement vers la droite si la valeur spécifiée est positive, vers la gauche si la valeur est négative.



Ce bloc change la position verticale (y) du lutin : une valeur positive le fait monter, tandis qu'une valeur négative le fait descendre sur la scène.



Exercice

- Programmer un lutin pour qu'il se déplace en carré de côté 100 pas en utilisant les 3 blocs demandés.
- Programmer un S en avançant par bon de 50 pas et en prenant une pause d'une seconde à chaque bond. Chaque ligne horizontale et verticale équivaut à 100 pas

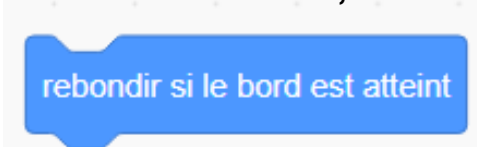
Orientation et rotation

- Ce bloc fixe l'orientation du lutin lorsqu'il doit changer de direction, par exemple en touchant un bord, en suivant la souris ou un autre lutin. Trois modes d'orientation sont possibles, mais le plus courant est le mode gauche-droite, qui donne un effet réaliste aux déplacements.



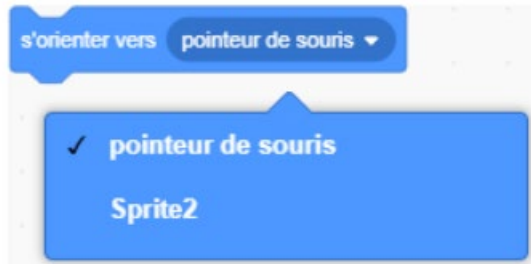
360°

Noter que si vous voulez faire rebondir le lutin quand il touche le bord, il faut utiliser le bloc suivant:

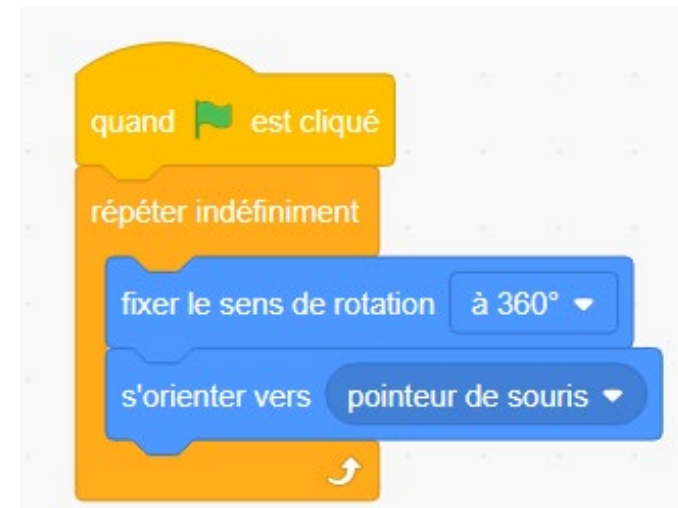


Orientation - Continuation

- Ce bloc oriente le lutin soit vers le pointeur de la souris, soit vers un autre lutin choisi dans la liste déroulante des éléments du projet.

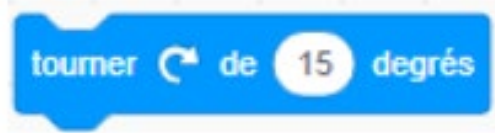


- Testez ces deux programmes suivants:



Rotation

Ce bloc permet de faire pivoter le lutin dans le sens des aiguilles d'une montre, selon un nombre de degrés que l'on peut définir.



Ce bloc permet de faire pivoter le lutin dans le sens opposé des aiguilles d'une montre, selon un nombre de degrés que l'on peut définir.



Ce bloc définit la direction dans laquelle le lutin doit se déplacer. Lorsque vous ouvrez la zone de saisie, un cadran circulaire apparaît, permettant de choisir l'orientation avec la souris : vers le haut (0°), vers le bas (180°), vers la droite (90°) ou vers la gauche (-90°).



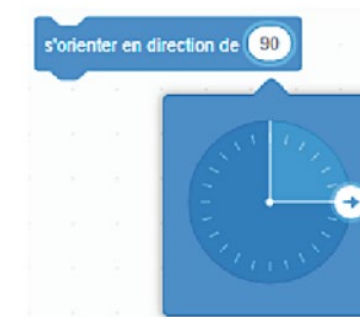
Pour se diriger vers le haut



Pour se diriger vers le bas

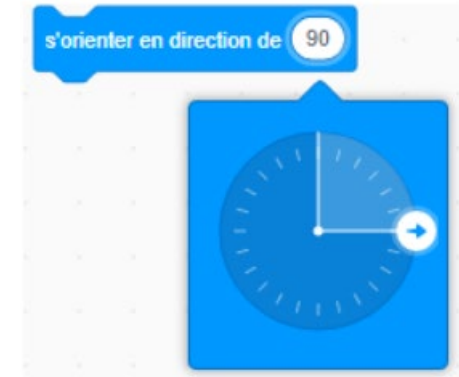


Pour se diriger vers la gauche



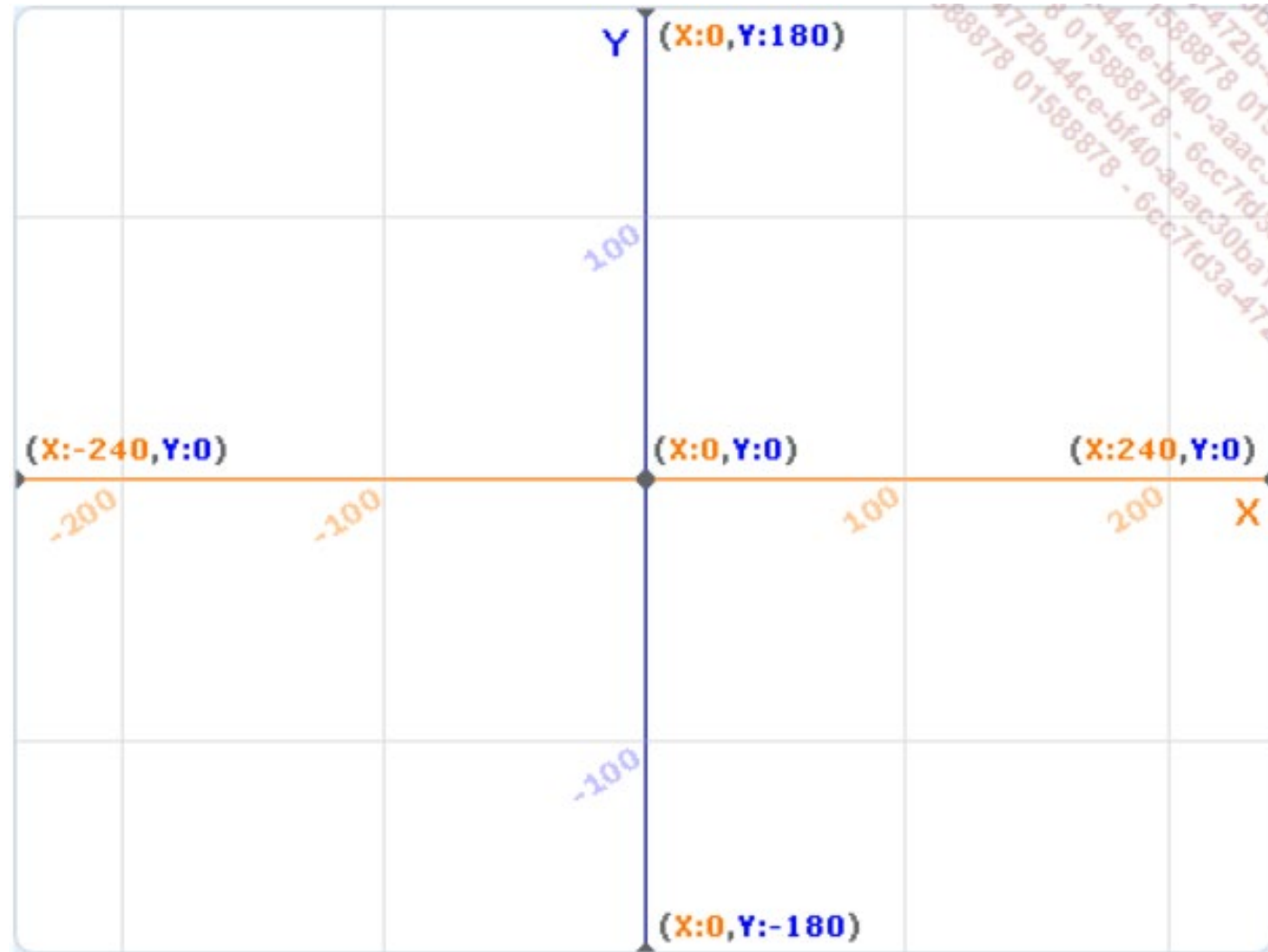
Pour se diriger vers la droite

Il est aussi possible d'indiquer d'autres angles pour créer des déplacements particuliers, comme le rebond d'une balle.



Les déplacements absolus

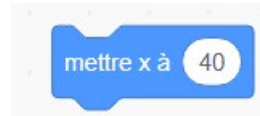
- Les déplacements en valeurs absolues permettent de positionner un lutin à un endroit précis de la scène grâce à des coordonnées x et y.
- La scène mesure 480 pas de large et 360 pas de haut, avec pour centre le point (x = 0, y = 0).
- Les coordonnées sont positives vers la droite et vers le haut, et négatives vers la gauche et vers le bas.



Quatre blocs permettent de positionner le lutin sur la scène en se basant sur des coordonnées x et y précises.



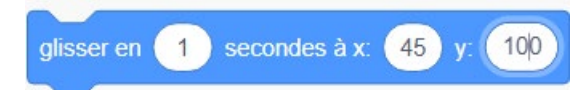
le lutin se met à la position (0, 180)



Ce bloc sert à déplacer le lutin horizontalement sur l'axe des abscisses en fonction d'une valeur comprise entre -240 et +240. Lors de son utilisation, l'ancienne position x est remplacée par la nouvelle valeur, par exemple 40.



Ce bloc sert à déplacer le lutin verticalement sur l'axe des ordonnées en fonction d'une valeur comprise entre -180 et +180. Lors de son utilisation, l'ancienne position y est remplacée par la nouvelle valeur, par exemple 180.



Contrairement aux autres blocs, celui-ci permet de voir le lutin se déplacer sur la scène. Le lutin glisse en douceur vers les coordonnées indiquées, et la vitesse du mouvement dépend de la durée choisie : plus celle-ci est longue, plus le déplacement est lent.

Autres blocs de mouvement

Le lutin concerné par ce bloc se déplace en glissant vers :

- une position aléatoire sur la scène ;
- le pointeur de souris ;



Le lutin concerné par ce bloc peut se positionner :

- d'une manière aléatoire sur la scène ;
- aux coordonnées x et y du pointeur de souris ;

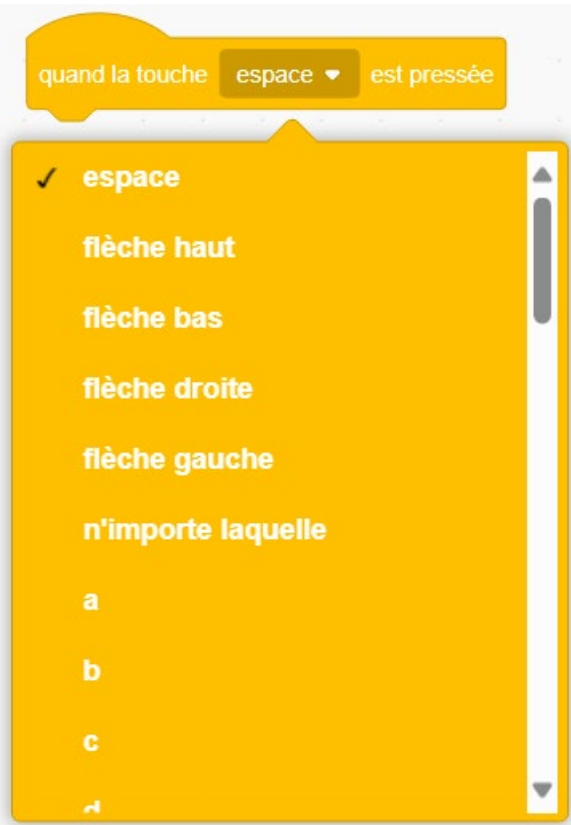


Les blocs Événements

- Un programme se compose d'une série d'instructions qui sont lues et exécutées les unes après les autres, un peu comme les étapes d'une recette de cuisine. U
- ne procédure correspond à une partie de ce programme : elle contient des instructions destinées à déclencher des actions précises.
- Lorsque plusieurs sprites et arrière-plans sont utilisés dans un même projet, il est nécessaire de créer des procédures propres à chacun, ainsi que des procédures permettant de coordonner leurs actions entre eux. Pour cela, on utilise les blocs des catégories Événements et Contrôle.



Ce bloc, placé au début du programme, déclenche l'exécution des instructions lorsqu'on clique sur le drapeau vert, qui sert de bouton de démarrage.



Ce bloc exécute l'instruction attachée lorsqu'une touche spécifique du clavier, choisie dans le menu déroulant, est pressée. On peut sélectionner des touches directionnelles, alphabétiques ou numériques.

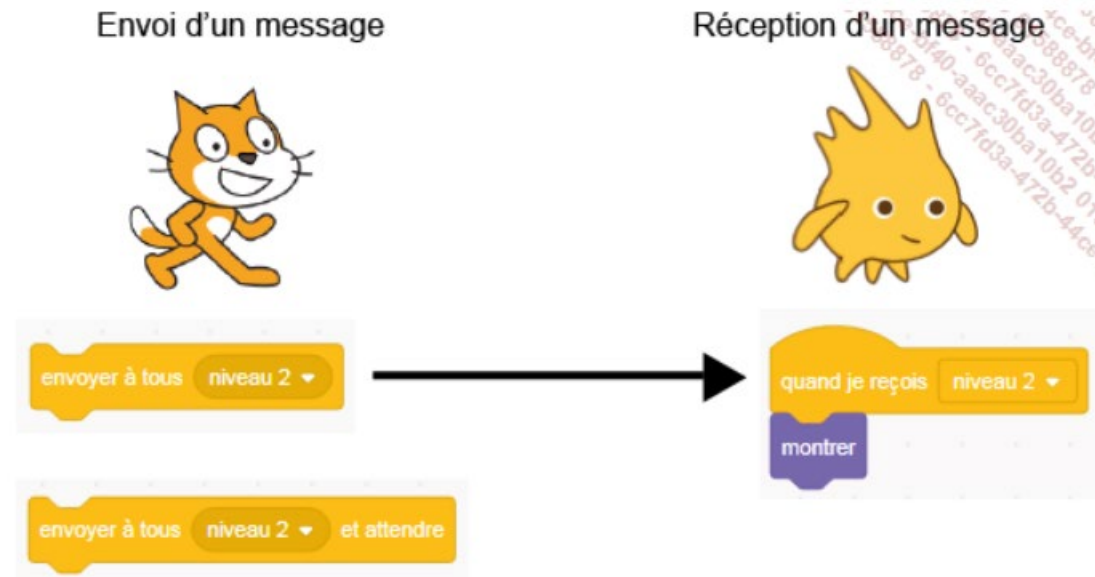
Exercice

- Quand on clique sur la touche « vers l'avant » du clavier, il faut faire avancer le lutin de 10 pas vers l'avant
- Quand on clique sur la touche « arrière » du clavier, il faut orienter le lutin vers l'arrière et avancer de 10 pas
- Quand on clique sur la touche « flèche vers le haut », il faut orienter le lutin vers le haut et avancer de 10 pas vers le haut.
- Quand on clique sur la touche « flèche vers le bas », il faut orienter le lutin vers le bas et avancer de 10 pas vers le bas.



Utiliser les messages

- Les messages servent à déclencher des actions, qu'elles soient communes ou propres à certains lutins ou arrière-plans.
- Un même message peut provoquer des réactions différentes selon les lutins ou affecter également les arrière-plans.
- Seuls les lutins munis d'un bloc de réception réagissent à un message envoyé ; les autres restent inactifs.



Exercice

- 🎮 Jeu 1 : Attrape le fantôme
- Objectif : Le joueur contrôle un lutin (ex. un chat) avec les flèches pour attraper un fantôme qui se déplace aléatoirement.
- Blocs utilisés :
 - quand touche flèche gauche pressée → ajouter -10 à x
 - quand touche flèche droite pressée → ajouter 10 à x
 - quand touche flèche haut pressée → ajouter 10 à y
 - quand touche flèche bas pressée → ajouter -10 à y
 - quand je reçois [attraper] → se cacher (pour le fantôme)
- Événements :
 - si le chat touche le fantôme → envoyer un message au fantôme pour qu'il disparaisse et se repositionne ailleurs.

Les blocs Contrôle

- Certains blocs servent à contrôler l'exécution du programme : ils permettent de mettre en pause, d'arrêter un script précis ou l'ensemble des programmes.

Ce bloc de contrôle met le programme en pause pendant un temps spécifié avant de continuer l'exécution des instructions suivantes.



Boucles

- Dans un programme, certaines instructions doivent se répéter. Pour simplifier le code et éviter les redondances, on utilise des boucles de répétition. Ces blocs, en forme de parenthèse, entourent les instructions à répéter, soit un nombre de fois déterminé, soit indéfiniment, jusqu'à ce qu'une condition mette fin à leur exécution.



Il s'agit d'une boucle de répétition dont le contenu s'exécute un nombre de fois déterminé. Une fois ce nombre atteint, le programme ne se répète plus.



Il s'agit d'une boucle de répétition dont le contenu s'exécute indéfiniment, ou jusqu'à ce qu'une condition mette fin à son exécution. Ce bloc est toujours utilisé avec des conditions afin que celles-ci soient continuellement vérifiées.

Exercice: Lutin rebondissant et changeant de taille

- Contexte
 - Tu vas créer un lutin qui bouge automatiquement sur la scène et peut changer de taille selon les touches que l'utilisateur appuie.
- Objectifs
 - Faire déplacer le lutin de gauche à droite indéfiniment.
 - Lorsqu'il atteint le bord de la scène, il rebondit automatiquement.
 - L'utilisateur peut appuyer sur des touches pour modifier la taille du lutin :
 - Une touche pour réduire la taille
 - Une autre touche pour augmenter la taille
- Consignes
 - Utiliser les blocs de Mouvement : avancer, rebondir si sur le bord, changer taille.
 - Utiliser les blocs d'Événements : quand drapeau vert cliqué, quand touche pressée.
 - Utiliser une boucle répétée indéfiniment pour faire bouger le lutin.
 - Choisir deux touches pour changer la taille du lutin (par exemple haut et bas).

Les conditions

- Les blocs conditionnels exécutent les instructions qui leur sont attachées uniquement si la condition est remplie. Avant chaque exécution, le programme vérifie si la condition est vraie ou fausse, il s'agit de conditions de type **booléen**.
- Les blocs servant à créer ces conditions possèdent des espaces dans lesquels d'autres blocs, notamment issus des catégories **Capteurs** et **Opérateurs**, peuvent être insérés pour définir la condition. Ces blocs sont reconnaissables à leurs extrémités pointues et sont appelés blocs à valeurs.



Si la condition spécifiée est vraie, le programme à l'intérieur du bloc s'exécute. Si elle est fausse, le contenu du bloc est ignoré et le script continue.

Lorsqu'elle est utilisée seule, sans boucle de répétition, la condition n'est vérifiée qu'une seule fois. C'est pourquoi il est conseillé de placer les conditions à l'intérieur d'une boucle.



Si la condition est vraie, le programme dans la première partie du bloc s'exécute. Si elle est fausse, le programme dans la partie « sinon » est exécuté.

Les blocs Capteurs

- Les blocs de type Capteurs servent à créer des conditions et à permettre des interactions entre les différents lutins d'un programme.
- Les blocs à valeurs aux extrémités pointues sont des blocs booléens utilisés pour définir ces conditions.
- En informatique, une condition booléenne correspond à une variable pouvant être dans l'état « vrai » ou « faux ».
- Grâce à leur forme, ces blocs peuvent être insérés dans d'autres blocs.



Ce bloc renvoie « vrai » si l'élément choisi dans le menu déroulant, comme le pointeur de la souris ou le bord de la scène, est touché.

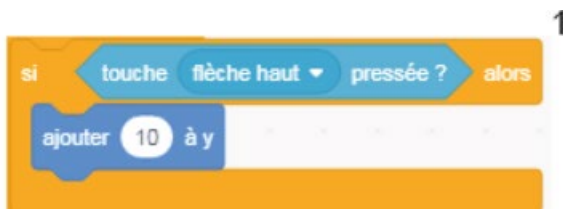
Il est souvent utilisé avec un bloc conditionnel pour vérifier si le lutin touche quelque chose comme le pointeur de la souris, le bord de la scène ou un autre lutin.



Ce petit programme permet d'avancer de 10 pas vers la droite quand on clique sur le bouton "flèche vers la droite" du clavier. Si le lutin touche le bord, le chat miaule



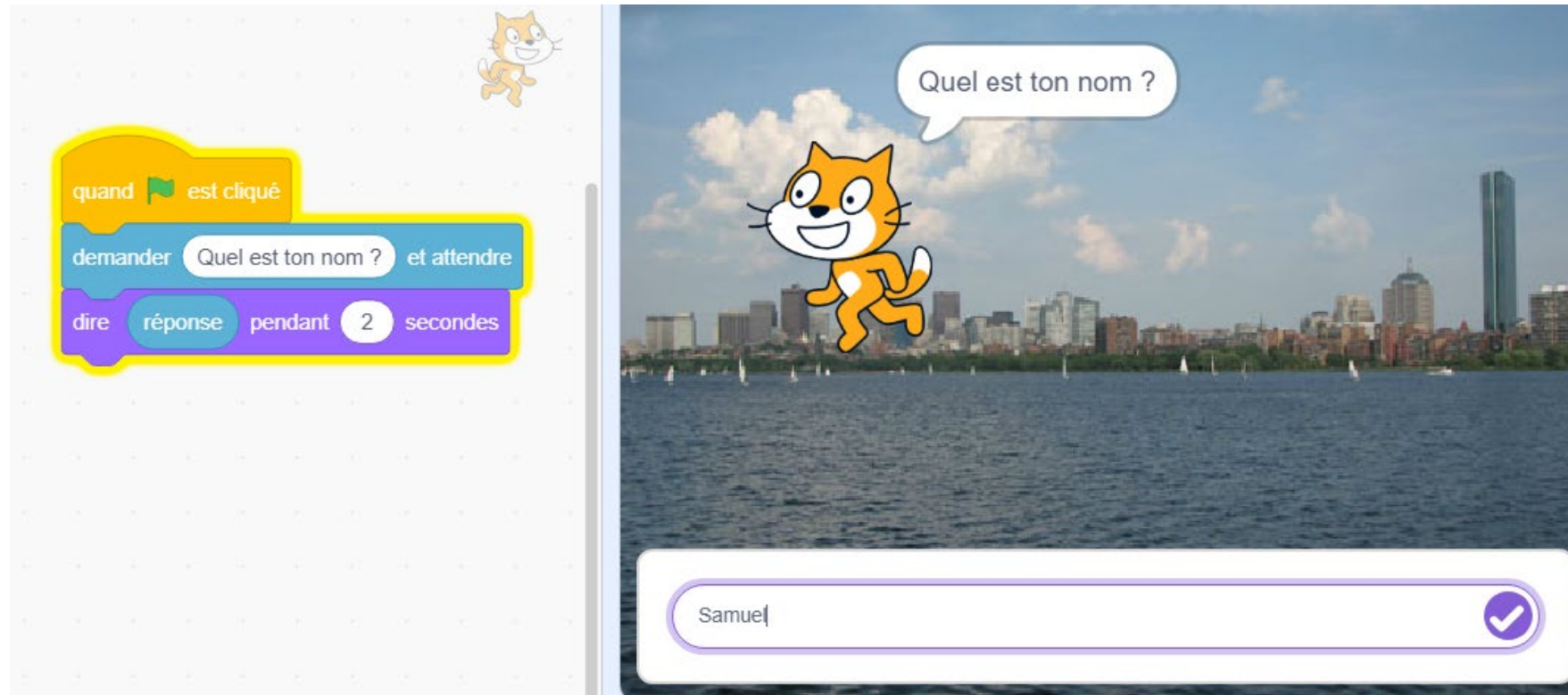
Ce bloc, à la fois de détection et de type booléen, permet de vérifier si une touche précise du clavier est enfoncée. Si le résultat est « vrai », l'action associée est exécutée. Le menu déroulant offre le choix entre des touches alphabétiques, numériques ou directionnelles.



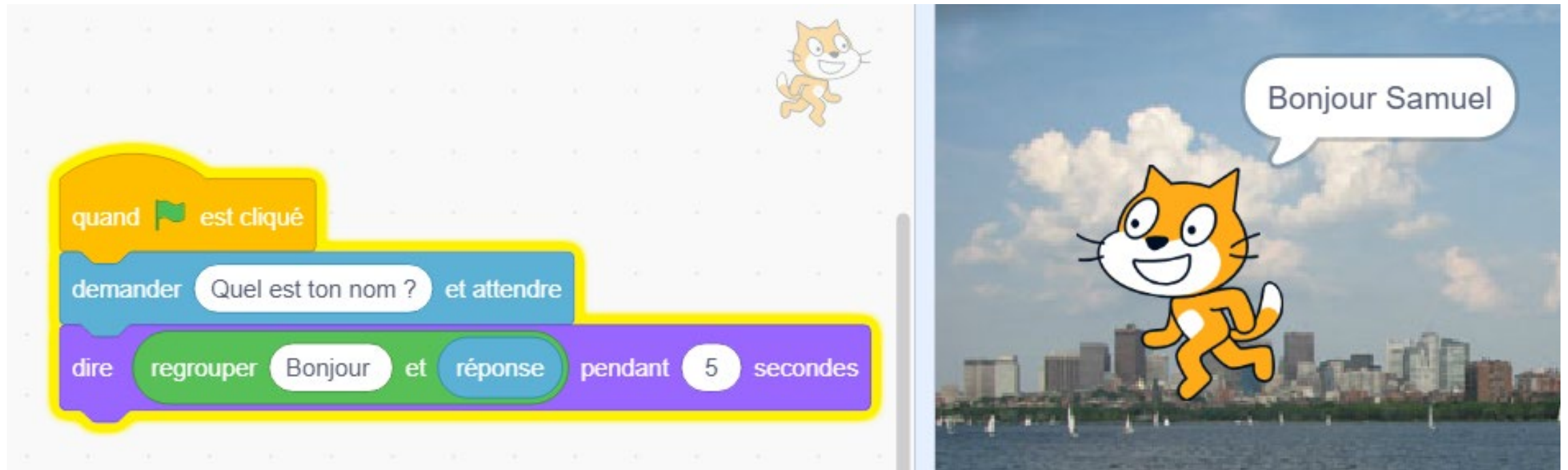
Ils sont utilisés pour déclencher des actions dans les jeux, comme le déplacement du lutin contrôlé ou les tirs.

Ils permettent aussi de lancer une partie non pas avec le bloc “quand drapeau vert est cliqué”, mais grâce à l’envoi d’un message spécifique.

Créer un dialogue



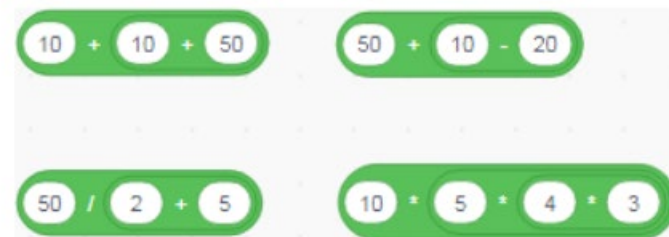
Le bloc **réponse** peut également être associé au bloc **regrouper ()** et **()** situé dans la catégorie **Opérateurs**. Il permet d'intégrer la réponse apportée par l'utilisateur dans une phrase.



Les blocs Opérateurs

- Un programme peut avoir besoin d'effectuer des calculs, qu'il s'agisse d'opérations mathématiques, de calculs géométriques ou de comparaisons de valeurs.
- Les blocs Opérateurs, reconnaissables à leur forme arrondie, s'insèrent dans les zones de saisie d'autres blocs et peuvent aussi s'emboîter entre eux.
- Ils sont utiles pour réaliser des opérations mathématiques, mais aussi pour développer des projets liés aux jeux.

Les blocs mathématiques



Le résultat de l'opération peut être dit par le sprite :





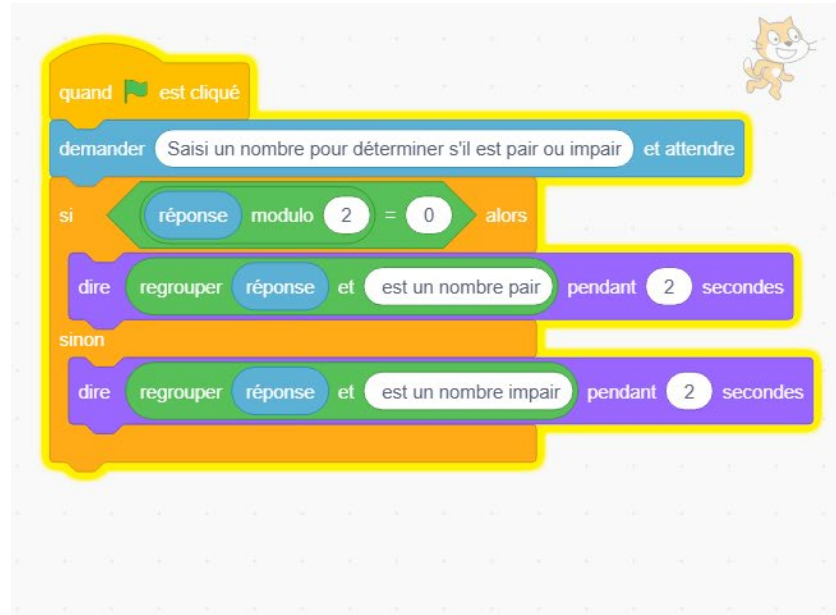
Ce bloc s'insère dans les zones de saisie des autres blocs. Il sert à sélectionner au hasard un nombre situé dans la fourchette des valeurs spécifiées.



Le lutin change de couleur après chaque seconde et la valeur de la couleur est déterminée de façon aléatoire

10 modulo 3

Le modulo est le reste de la division. Par exemple si le modulo d'un nombre est égal à 0, c'est un nombre pair. Dans le cas contraire, c'est un nombre impair.



Blocs de comparaison – arithmétiques



supérieur à



score supérieur à 0



inférieur à



score inférieur à 0

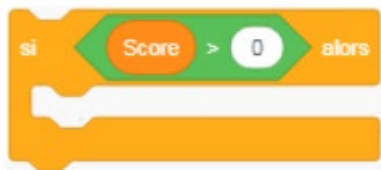
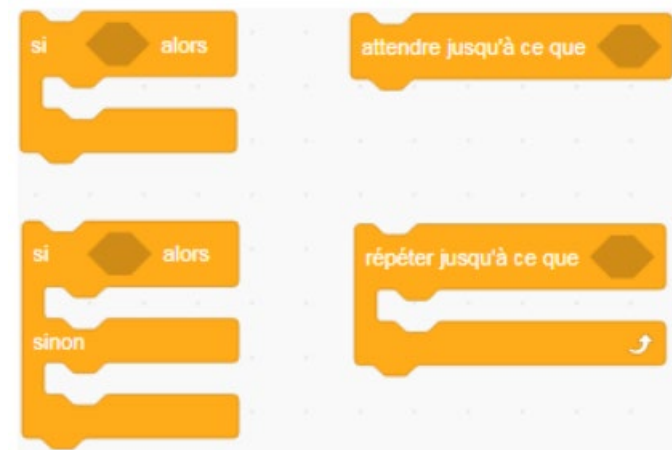


égal à

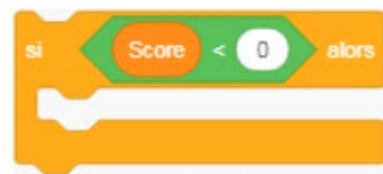


score égal à 0

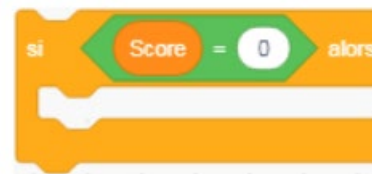
Pour définir des conditions booléennes, les blocs de comparaison arithmétique s'insèrent dans les blocs de la catégorie Contrôle.



Ce bloc renvoie « vrai » si la première valeur est plus grande que la seconde (par exemple, si la variable Score est supérieure à 0). Dans ce cas, le programme contenu dans le bloc s'exécute.



Ce bloc renvoie « vrai » si la première valeur est plus petite que la seconde (par exemple, si la variable Score est inférieure à 0). Dans ce cas, le programme à l'intérieur du bloc s'exécute.



Ce bloc renvoie « vrai » si la première valeur est égale à la seconde (par exemple, si la variable Score est égale à 0). Dans ce cas, le programme contenu dans le bloc s'exécute.

Comparaisons non mathématiques - opérateurs logiques



Ce bloc peut accueillir deux conditions booléennes. Le programme associé ne s'exécute que si les deux conditions sont vraies.



Ce bloc contient deux conditions booléennes. Le programme s'exécute si au moins l'une des deux est vraie.



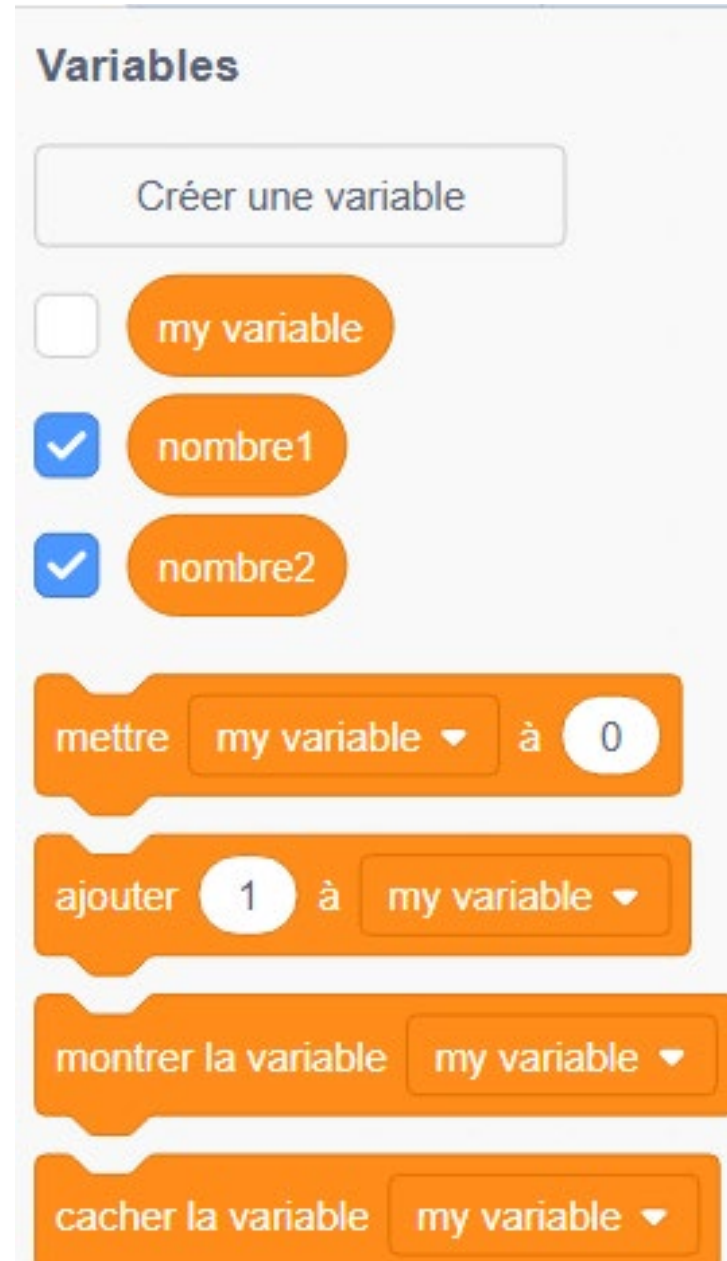
Contrairement aux blocs précédents, ce bloc renvoie « vrai » lorsque la condition n'est pas remplie, et « faux » lorsqu'elle l'est.

Exercices

- 🎯 Exercice 1 – Nombre pair ou impair
 - Quand on clique sur le drapeau vert, demande à l'utilisateur un nombre. Utilise l'opérateur modulo (nombre mod 2) pour vérifier s'il est pair ou impair. Affiche "Ton nombre est pair" ou "Ton nombre est impair".
 - 👉 Objectif : pratiquer l'opérateur modulo avec une condition.
- 🎯 Exercice 2 – Mot de passe simple
 - Quand on clique sur le drapeau vert, demande "Quel est le mot secret ?". Si le texte entré = "chat", dire "Accès autorisé". Sinon, dire "Accès refusé".
 - 👉 Objectif : utiliser l'opérateur = dans une condition.
- 🎯 Exercice 3 – Plus grand ou plus petit
 - Quand on clique sur le drapeau vert, demande un nombre. Si ce nombre est > 10 , dire "Grand nombre". Sinon, dire "Petit nombre".
 - 👉 Objectif : utiliser $>$ et $<$ avec une condition.
- 🎯 Exercice 4 – Jeu du nombre magique
 - Quand on clique sur le drapeau vert, demande "Choisis un nombre entre 1 et 5". Si le nombre est $= 3$, dire "Bravo, tu as trouvé !". Sinon, dire "Essaie encore".
 - 👉 Objectif : condition avec $=$.

Les blocs Variables

- Qu'il s'agisse d'une animation ou d'un jeu, un projet nécessite l'utilisation de données qui sont stockées et mises à jour au fil de l'exécution du programme.
- Ces données peuvent être **des variables** ou des listes.
- Une variable associe un nom à une valeur. Par exemple, la variable *nombre1* représente un nombre de points. Cette valeur n'est pas fixe : les variables sont dynamiques et peuvent changer au cours du temps.



nombre1

2

nombre2

9

Bienvenue au jeu de
calculatrice pour les
enfants



nombre1

2

nombre2

9

Je te pose des questions
d'addition et tu réponds



nombre1

2

nombre2

9

Appuie sur n'importe
quelle touche sur le clavier
pour débiter



nombre1

7

nombre2

8

Quel est le résultat de: $7 + 8$



nombre1

7

nombre2

8

Bravo, le résultat est bien
15





On essaie un mauvais résultat

nombre1

1

nombre2

7

Malheureusement, tu n'as pas trouvé la bonne réponse. La réponse attendue était 8

